EMPLOYEE RECORD STORAGE USING HASH TABLES

Master Project Report & Complete Java Console Implementation

Project Type	Data Structures / Console-Based Java Project
Core Structure	Hash Table + Separate Chaining
Collision Structure	Doubly Linked List
Key	Unique Employee ID
Operations	Insert, Search, Update, Delete, Display
Scalability	Dynamic Rehashing when load factor > 0.75
Future Extension	GUI can be added without rewriting the core DSA

1. Project Overview

Problem Statement: Design a console-based employee record management system that stores employee records using a hash table. Employee ID is the unique key. Collisions are resolved using separate chaining, with a doubly linked list used for each chain.

The project is intentionally DSA-focused. It does not require a backend or database. The hash-table core is kept independent from the console menu so a GUI can be added later.

2. Objectives

• Efficient employee record storage using hashing.

• Use employee ID as a unique search key.

• Resolve collisions using separate chaining.

• Use doubly linked lists for chains.

• Implement insert, search, update, delete and display.

• Prevent duplicate IDs.

• Validate input values.

• Maintain correct DLL prev/next links.

• Dynamically resize when load factor exceeds 0.75.

• Rehash all records safely without loss.

• Provide useful collision and load-factor statistics.

• Keep the DSA core reusable for a future GUI.

3. Modules

Module	Purpose
Employee	Complete employee record: ID, name, age, qualification, position and salary.
Node	Doubly linked-list node containing one Employee.
EmployeeHashTable	Hash table, chaining, CRUD, rehashing and statistics.
Hash Function	Division-method mapping from ID to bucket.
Separate Chaining	Stores colliding records in the same bucket chain.
Doubly Linked List	Maintains prev/next links in every chain.
Input	Safe console input and validation.
Main/Menu	Console-only user interface.

4. Employee Record Structure

Every node stores the complete employee record. The Employee ID is the key; name, age, qualification, position and salary are the associated data.

• ID: non-negative long integer, unique and immutable.

• Name: non-empty text.

• Age: 18–100 in this implementation.

• Qualification: non-empty text.

• Position: non-empty text.

• Salary: finite and non-negative.

5. Hashing Rule

The table uses the division method: **index = ID mod tableSize**. Therefore, with table size 5, ID 5 maps to bucket 0, not automatically to the fifth physical array position. This is a critical distinction in a hash table.

6. Separate Chaining + Doubly Linked List

If multiple IDs map to the same bucket, their nodes form a doubly linked list. For example, IDs 5, 10 and 15 can form 5 ■ 10 ■ 15 when they map to the same bucket. Searching ID 10 first calculates its bucket and then traverses that chain.

7. Dynamic Rehashing

The initial table size is 5, but it is not permanently fixed. When count/tableSize exceeds 0.75, a larger prime-sized table is created and every existing node is redistributed using the new table size. Employee IDs and record data remain unchanged.

8. CRUD Operations

Operation	Implementation
Insert	Validate → duplicate check → hash → add to chain → count → load-factor check.
Search	Hash ID → access bucket → traverse only that chain → return complete record.
Update	Find by immutable ID → validate → change employee data fields.
Delete	Find node → repair both DLL directions → remove node → decrement count.
Display	Traverse all buckets and chains and print complete records.
Statistics	Show table size, count, load factor, collisions, rehashes and maximum chain.

9. Complexity

Operation	Average	Worst Case
Insert	O(1)	O(n)
Search	O(1)	O(n)
Update	O(1)	O(n)
Delete	O(1)	O(n)
Rehash	O(n)	O(n)
Display	O(n)	O(n)

Average-case performance assumes a good hash distribution and controlled load factor. n is the number of records.

10. Loopholes & Corrections

Potential loophole	Correction
Duplicate IDs	Rejected before insertion.
ID changes	ID is immutable; change requires delete + reinsert.
Overloaded fixed table	Dynamic rehashing above 0.75.
Record loss during rehash	Every existing node is redistributed exactly once.
Broken DLL deletion	Head, middle and tail cases repair prev/next pointers.
Invalid data	ID, age, text and salary are validated.
Unnecessary full-table search	Search hashes the ID first.
GUI mixed with DSA	Core table logic is separate from the menu.

Potential loophole	Correction
Poor observability	Statistics expose load factor, collisions, rehashes and chain length.

11. Test Cases

Case	Expected result
Normal insert	Record stored correctly.
IDs 5, 10, 15	Collision chain handled.
Search existing	Complete details returned.
Search missing	Not found.
Duplicate ID	Rejected.
Delete head/middle/tail	DLL remains valid.
Delete only node	Bucket becomes empty safely.
Update	Data changes; ID stays unchanged.
Invalid age/salary/text	Rejected.
Rehash	All records remain accessible.
Large data	No data or structural loss.

12. 1,000,000-Record Java Stress Test

Metric	Result
Records inserted	1,000,000
Records remaining	995,000
Collisions detected	275,920
Rehash operations	18
Final table size	1,572,863
Search tests	10,000
Update tests	5,000
Delete tests	5,000 + repeated-delete checks
Validation	9 invalid + extreme valid + special text
DLL nodes verified	995,000
Overall result	ALL JAVA STRESS TESTS PASSED

The stress test used sequential, collision-heavy, very large and random ID distributions. It also verified duplicate protection, missing IDs, updates, repeated deletes, rehashing and both directions of the doubly linked lists.

13. Recommended Demo Flow

• Insert normal employee records.

• Insert deliberately colliding IDs.

• Search and display a complete record.

• Attempt a duplicate ID.

• Update details and show that the ID is unchanged.

• Delete head, middle and tail chain nodes.

• Show hash-table statistics.

• Insert enough records to trigger rehashing.

• Search records again after rehashing.

14. Future Enhancements

• JavaFX or another permitted GUI.

• File/CSV persistence.

• CSV import/export.

• JUnit automated tests.

• Performance visualization.

• Optional database integration if the academic scope later permits it.

Rule: future modules should call the existing EmployeeHashTable API instead of duplicating hashing logic.

15. Suggested Multi-File Organization

• Employee.java — employee data model.

• Node.java — doubly linked-list node.

• EmployeeHashTable.java — hashing, chaining, CRUD and rehashing.

• InputValidator.java — input validation.

• EmployeeRecordSystem.java — console menu and main method.

• EmployeeHashTableTest.java — automated tests.

The complete implementation included below is intentionally provided as one Java file for simple submission and execution.

16. Complete Java Code

Save this source as **EmployeeRecordSystem.java**.

```
import java.util.*;

public class EmployeeRecordSystem {

    // ==================== EMPLOYEE ====================
    static class Employee {
        private final long id;
        private String name;
        private int age;
        private String qualification;
        private String position;
        private double salary;

        Employee(long id, String name, int age, String qualification,
                 String position, double salary) {
            this.id = id;
            this.name = name;
            this.age = age;
            this.qualification = qualification;
            this.position = position;
            this.salary = salary;
        }

        long getId() { return id; }
        String getName() { return name; }
        int getAge() { return age; }
        String getQualification() { return qualification; }
        String getPosition() { return position; }
        double getSalary() { return salary; }

        void setName(String name) { this.name = name; }
        void setAge(int age) { this.age = age; }
        void setQualification(String qualification) {
            this.qualification = qualification;
        }
        void setPosition(String position) { this.position = position; }
        void setSalary(double salary) { this.salary = salary; }

        void display() {
            System.out.println("----------------------------------------");
            System.out.println("Employee ID   : " + id);
            System.out.println("Name          : " + name);
            System.out.println("Age           : " + age);
            System.out.println("Qualification : " + qualification);
            System.out.println("Position      : " + position);
            System.out.printf("Salary        : %.2f%n", salary);
```

```
        System.out.println("---------------------------------------");
    }
}

// ==================== DLL NODE ====================
static class Node {
    Employee employee;
    Node prev;
    Node next;

    Node(Employee employee) {
        this.employee = employee;
    }
}

// ==================== HASH TABLE ====================
static class EmployeeHashTable {
    private Node[] table;
    private int count;
    private long collisions;
    private int rehashCount;

    private static final double MAX_LOAD_FACTOR = 0.75;

    EmployeeHashTable(int initialSize) {
        if (initialSize < 3) {
            initialSize = 5;
        }
        table = new Node[nextPrime(initialSize)];
    }

    /*
     * Division-method hash.
     * floorMod also keeps the result valid for unusual long values.
     */
    private int hash(long id) {
        return (int) Math.floorMod(id, table.length);
    }

    int size() {
        return table.length;
    }

    int count() {
        return count;
    }

    long collisions() {
        return collisions;
    }

    int rehashCount() {
        return rehashCount;
    }

    double loadFactor() {
        return (double) count / table.length;
    }
```

```
// ---------------- INSERT ----------------
boolean insert(Employee employee) {
    if (!isValidEmployee(employee)) {
        return false;
    }

    if (search(employee.getId()) != null) {
        return false; // duplicate ID
    }

    int index = hash(employee.getId());
    Node newNode = new Node(employee);

    if (table[index] == null) {
        table[index] = newNode;
    } else {
        collisions++;

        Node current = table[index];
        while (current.next != null) {
            current = current.next;
        }

        current.next = newNode;
        newNode.prev = current;
    }

    count++;

    if (loadFactor() > MAX_LOAD_FACTOR) {
        rehash();
    }

    return true;
}

// ---------------- SEARCH ----------------
Employee search(long id) {
    Node current = table[hash(id)];

    while (current != null) {
        if (current.employee.getId() == id) {
            return current.employee;
        }
        current = current.next;
    }

    return null;
}

// ---------------- UPDATE ----------------
boolean update(long id, String name, int age,
               String qualification, String position,
               double salary) {

    Employee employee = search(id);

    if (employee == null || !isValidDetails(name, age,
            qualification, position, salary)) {
        return false;
    }

    employee.setName(name);
    employee.setAge(age);
    employee.setQualification(qualification);
    employee.setPosition(position);
    employee.setSalary(salary);

    return true;
}

/*
 * Employee ID is immutable because it is the hash key.
 * If an ID must change, delete the old record and insert
 * a new record using the new ID.
 */

// ---------------- DELETE ----------------
boolean delete(long id) {
    int index = hash(id);
    Node current = table[index];

    while (current != null) {
        if (current.employee.getId() == id) {

            if (current.prev != null) {
                current.prev.next = current.next;
            } else {
                table[index] = current.next;
            }

            if (current.next != null) {
                current.next.prev = current.prev;
            }

            current.prev = null;
            current.next = null;
            count--;

            return true;
        }

        current = current.next;
    }
```

```
    return false;
}

// ---------------- DISPLAY ----------------
void displayAll() {
    if (count == 0) {
        System.out.println("No employee records found.");
        return;
    }

    for (int i = 0; i < table.length; i++) {
        Node current = table[i];

        if (current != null) {
            System.out.println("\nBucket [" + i + "]");

            while (current != null) {
                current.employee.display();
                current = current.next;
            }
        }
    }
}

// ---------------- STATISTICS ----------------
void displayStatistics() {
    int nonEmpty = 0;
    int maxChain = 0;

    for (Node head : table) {
        if (head != null) {
            nonEmpty++;
            int length = 0;
            Node current = head;

            while (current != null) {
                length++;
                current = current.next;
            }

            maxChain = Math.max(maxChain, length);
        }
    }

    System.out.println("\n========== HASH TABLE STATISTICS ==========");
    System.out.println("Table size       : " + table.length);
    System.out.println("Employee count   : " + count);
    System.out.printf("Load factor      : %.4f%n", loadFactor());
    System.out.println("Non-empty buckets: " + nonEmpty);
    System.out.println("Collisions       : " + collisions);
    System.out.println("Rehash count     : " + rehashCount);
    System.out.println("Maximum chain    : " + maxChain);
    System.out.println("============================================");
}

// ---------------- REHASH ----------------
private void rehash() {
    Node[] oldTable = table;

    int newSize = nextPrime(oldTable.length * 2 + 1);
    table = new Node[newSize];

    /*
     * Reinsert every existing node into the new table.
     * Collision count is intentionally not incremented here:
     * it represents collisions caused by user insertions,
     * while rehashing is a maintenance operation.
     */
    for (Node head : oldTable) {
        Node current = head;

        while (current != null) {
            Node next = current.next;

            current.prev = null;
            current.next = null;

            int index = hash(current.employee.getId());

            if (table[index] == null) {
                table[index] = current;
            } else {
                Node tail = table[index];

                while (tail.next != null) {
                    tail = tail.next;
                }

                tail.next = current;
                current.prev = tail;
            }

            current = next;
        }
    }

    rehashCount++;
}

// ---------------- VALIDATION ----------------
private static boolean isValidEmployee(Employee employee) {
    if (employee == null || employee.getId() < 0) {
        return false;
    }
```

```
        return isValidDetails(
                employee.getName(),
                employee.getAge(),
                employee.getQualification(),
                employee.getPosition(),
                employee.getSalary()
        );
    }

    private static boolean isValidDetails(
            String name, int age, String qualification,
            String position, double salary) {

        return name != null && !name.trim().isEmpty()
                && age >= 18 && age <= 100
                && qualification != null
                && !qualification.trim().isEmpty()
                && position != null
                && !position.trim().isEmpty()
                && Double.isFinite(salary)
                && salary >= 0;
    }

    // ---------------- PRIME SIZE ----------------
    private static int nextPrime(int number) {
        if (number < 3) {
            return 3;
        }

        while (!isPrime(number)) {
            number++;
        }

        return number;
    }

    private static boolean isPrime(int number) {
        if (number < 2) {
            return false;
        }

        if (number % 2 == 0) {
            return number == 2;
        }

        for (int i = 3; (long) i * i <= number; i += 2) {
            if (number % i == 0) {
                return false;
            }
        }

        return true;
    }
}

// ==================== INPUT ====================
static class Input {
    static String readNonEmpty(Scanner sc, String prompt) {
        while (true) {
            System.out.print(prompt);
            String value = sc.nextLine().trim();

            if (!value.isEmpty()) {
                return value;
            }

            System.out.println("Input cannot be empty.");
        }
    }

    static long readId(Scanner sc) {
        while (true) {
            try {
                System.out.print("Employee ID: ");
                long id = Long.parseLong(sc.nextLine().trim());

                if (id >= 0) {
                    return id;
                }

                System.out.println("ID cannot be negative.");
            } catch (NumberFormatException e) {
                System.out.println("Enter a valid integer ID.");
            }
        }
    }

    static int readAge(Scanner sc) {
        while (true) {
            try {
                System.out.print("Age: ");
                int age = Integer.parseInt(sc.nextLine().trim());

                if (age >= 18 && age <= 100) {
                    return age;
                }

                System.out.println("Age must be between 18 and 100.");
            } catch (NumberFormatException e) {
                System.out.println("Enter a valid age.");
            }
        }
    }

    static double readSalary(Scanner sc) {
```

```
        while (true) {
            try {
                System.out.print("Salary: ");
                double salary = Double.parseDouble(sc.nextLine().trim());

                if (Double.isFinite(salary) && salary >= 0) {
                    return salary;
                }

                System.out.println("Enter a valid non-negative salary.");
            } catch (NumberFormatException e) {
                System.out.println("Enter a valid salary.");
            }
        }
    }
}

// ==================== MAIN ====================
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    EmployeeHashTable table = new EmployeeHashTable(5);

    while (true) {
        System.out.println("\n========== EMPLOYEE RECORD STORAGE ==========");
        System.out.println("1. Insert Employee");
        System.out.println("2. Search Employee");
        System.out.println("3. Update Employee");
        System.out.println("4. Delete Employee");
        System.out.println("5. Display All Employees");
        System.out.println("6. Hash Table Statistics");
        System.out.println("7. Exit");
        System.out.println("============================================");

        System.out.print("Enter choice: ");
        String choice = sc.nextLine().trim();

        switch (choice) {
            case "1":
                insertEmployee(sc, table);
                break;

            case "2":
                searchEmployee(sc, table);
                break;

            case "3":
                updateEmployee(sc, table);
                break;

            case "4":
                deleteEmployee(sc, table);
                break;

            case "5":
                table.displayAll();
                break;

            case "6":
                table.displayStatistics();
                break;

            case "7":
                sc.close();
                System.out.println("Program terminated.");
                return;

            default:
                System.out.println("Invalid choice.");
        }
    }
}

static void insertEmployee(Scanner sc, EmployeeHashTable table) {
    long id = Input.readId(sc);

    if (table.search(id) != null) {
        System.out.println("Employee ID already exists.");
        return;
    }

    String name = Input.readNonEmpty(sc, "Name: ");
    int age = Input.readAge(sc);
    String qualification =
            Input.readNonEmpty(sc, "Qualification: ");
    String position =
            Input.readNonEmpty(sc, "Position: ");
    double salary = Input.readSalary(sc);

    Employee employee =
            new Employee(id, name, age, qualification, position, salary);

    if (table.insert(employee)) {
        System.out.println("Employee inserted successfully.");
    } else {
        System.out.println("Employee insertion failed.");
    }
}

static void searchEmployee(Scanner sc, EmployeeHashTable table) {
    long id = Input.readId(sc);
    Employee employee = table.search(id);

    if (employee != null) {
        employee.display();
    } else {
```

```
            System.out.println("Employee not found.");
        }
    }

    static void updateEmployee(Scanner sc, EmployeeHashTable table) {
        long id = Input.readId(sc);

        if (table.search(id) == null) {
            System.out.println("Employee not found.");
            return;
        }

        String name = Input.readNonEmpty(sc, "New Name: ");
        int age = Input.readAge(sc);
        String qualification =
                Input.readNonEmpty(sc, "New Qualification: ");
        String position =
                Input.readNonEmpty(sc, "New Position: ");
        double salary = Input.readSalary(sc);

        if (table.update(id, name, age, qualification, position, salary)) {
            System.out.println("Employee updated successfully.");
        } else {
            System.out.println("Employee update failed.");
        }
    }

    static void deleteEmployee(Scanner sc, EmployeeHashTable table) {
        long id = Input.readId(sc);

        if (table.delete(id)) {
            System.out.println("Employee deleted successfully.");
        } else {
            System.out.println("Employee not found.");
        }
    }
}
```

17. Compilation & Execution

Compile: javac EmployeeRecordSystem.java
Run: java EmployeeRecordSystem

Only standard Java libraries are used. No database, backend server, external package or internet connection is required.

18. Final Submission Checklist

• Hash table implemented.

• Employee ID used as unique key.

• Separate chaining implemented.

• Doubly linked list used for chains.

• Complete employee data stored.

• Duplicate IDs rejected.

• Insert/Search/Update/Delete implemented.

• Input validation implemented.

• Dynamic rehashing implemented.

• Load factor monitored.

• DLL deletion cases handled.

• Statistics included.

• Console-based design maintained.

• Core logic separated from interface.

• Future GUI integration possible.

• 1,000,000-record Java stress test completed successfully.

Conclusion: The project demonstrates hashing, collision resolution, doubly linked lists, dynamic resizing, CRUD operations, validation, complexity analysis and large-scale testing while remaining appropriate for a console-based DSA project.